VesselOps 시스템 아키텍처 설계서

항목	내용
문서명	선박 기관 원격 상태감시·예지보전 플랫폼 아키텍처 설계서
버전	v1.5
작성일	2026-10-16
작성자	최승렬
선행문서	`01-requirements.md` (v1.3)
관련문서	`00-glossary.md` (용어 정의), `03-interface-spec.md` (인터페이스 규격), `06-function-spec.md` (프로그램 설계)

목차

1. 개요
2. 논리 아키텍처
3. 컴포넌트 구성
4. 물리 배치 및 네트워크 경계
5. 데이터 흐름 — 정상 수집 / 통신 단절·복구 / **적응형 전송**
6. 기술 선정 근거 — 대안 비교 / 라이선스 검토
7. 보안 설계
8. 확장성·가용성
9. **주요 설계 결정 기록 (ADR)** — ADR-001 ~ 011
10. 제약 및 가정
11. 변경 이력

> 선박·자동화 용어는 `00-glossary.md` 를 참조한다.

1. 개요

본 문서는 요구사항정의서에 정의된 기능·비기능 요구사항을 만족하는 시스템 구조를 정의한다. 구성요소의 책임 분리, 통신 방식, 배치 형태, 그리고 **왜 그렇게 결정했는지**를 기록한다.

설계의 출발점이 된 제약은 세 가지다.

제약	설계에 미친 영향
선박-육상 통신은 저대역·종량제·간헐 단절	Edge 측 필터링·버퍼링에 더해 **적응형 전송·Pre-trigger 링버퍼** 필요 (FR-103·104·108·109)

제약	설계에 미친 영향
본선 자동화망은 폐쇄망, 제어계 접근 불가	읽기 전용 수집, 감시계에만 연결 (FR-107)
무료 클라우드 자원 1대로 운영	단일 VM · 컨테이너 구성, 쿠버네티스 미도입

2. 논리 아키텍처



2.1 계층 분리 원칙

- **상위 계층은 하위 계층만 호출한다.** 표현 계층이 DB에 직접 붙지 않는다.
- **수집 계층과 업무 계층은 서로를 호출하지 않는다.** DB를 통해서만 만난다. 둘 중 하나가 죽어도 나머지가 계속 동작한다.
- **설비 계층 교체가 상위에 영향을 주지 않는다.** 시뮬레이터 → 실선 AMS 교체 시 Edge 수집기의 엔드포인트 설정만 바뀐다.

3. 컴포넌트 구성

#	컴포넌트	기술	책임	관련 요구사항
1	`sim-fleet`	Node.js + node-opcua	선박 4척 엔진 물리모델 시뮬레이션, **OPC UA 서버**로 노출, 고장 주입 API	2.2 제외범위
2	`edge-gw`	Node.js + node-opcua	**OPC UA 클라이언트**. 구독·Deadband 필터·**적응형 전송 판정**·**Pre-trigger 링버퍼**·우선순위 큐·MQTT 발행	FR-101~110

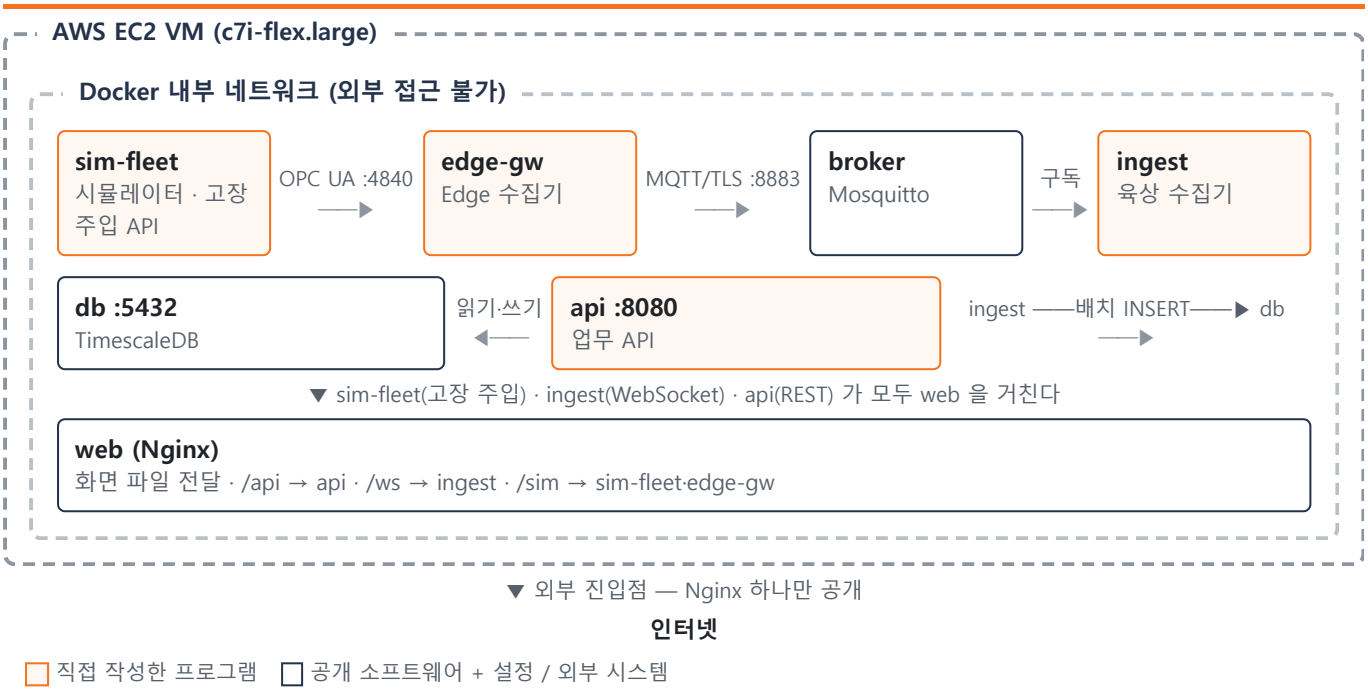
#	컴포넌트	기술	책임	관련 요구사항
3	`broker`	Eclipse Mosquitto	MQTT 메시지 중계, QoS 1, 세션 보존	FR-106, FR-151
4	`ingest`	Node.js	MQTT 구독, 스키마 검증, 배치 적재, WebSocket 푸시	FR-151~156
5	`db`	PostgreSQL 16 + TimescaleDB	마스터·시계열 저장, 연속집계, 압축·보존	NFR-03, NFR-04
6	`api`	Java 21 / Spring Boot 3.5	인증·권한, 마스터관리, 알람 룰 엔진, 성능지표	FR-200~500
7	`web`	Nginx + 정적 대시보드	화면 제공, 리버스 프록시, HTTPS 종단	FR-600

3.1 컴포넌트 간 통신

출발	도착	프로토콜	방향	비고
`edge-gw`	`sim-fleet`	OPC UA (4840)	읽기 전용	실선에서는 AMS
`edge-gw`	`broker`	MQTT over TLS (8883)	발행	선박별 클라이언트 인증서 (ADR-011)
`ingest`	`broker`	MQTT over TLS (8883)	구독	클라이언트 인증서
`ingest`	`db`	PostgreSQL 프로토콜	쓰기	배치 INSERT
`ingest`	브라우저	WebSocket	푸시	`web` 프록시 경유
`api`	`db`	PostgreSQL 프로토콜	읽기·쓰기	Spring `JdbcClient` (ADR-008)
브라우저	`sim-fleet` · `edge-gw`	HTTP/REST	요청	시연 제어. `web` 프록시 경유
브라우저	`api`	HTTPS/REST	요청	`web` 프록시 경유

---

4. 물리 배치 및 네트워크 경계



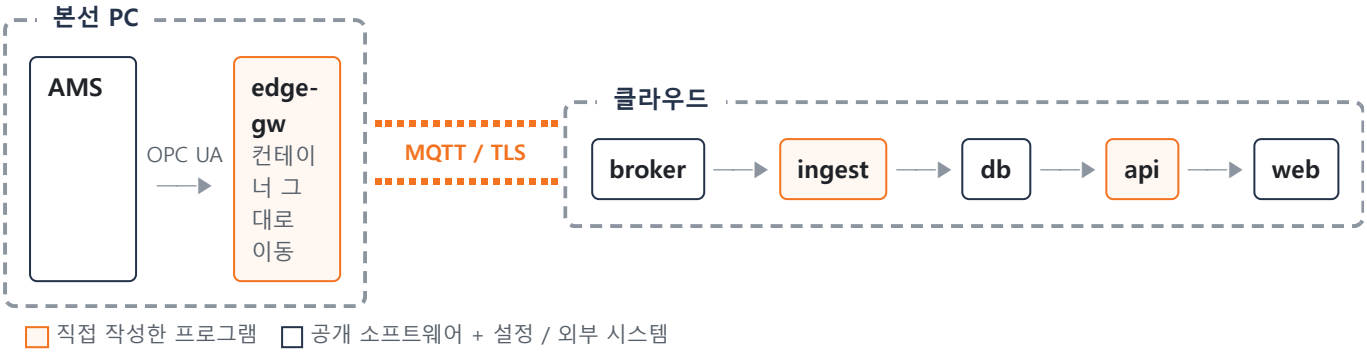
4.1 포트 공개 정책

포트	서비스	외부 공개	근거
443	HTTPS (Nginx)	✅ 공개	유일한 외부 진입점
80	HTTP	✅ 공개	443 으로 리다이렉트만
22	SSH	🔒 특정 IP 만	운영 접속
4840	OPC UA	❌ 차단	**컨테이너 간 통신 전용**
8883	MQTT over TLS	❌ 차단 (기본)	컨테이너 간 통신. 본선을 다른 장비로 분리할 때만 공개하며, 그때도 클라이언트 인증서 없이는 접속되지 않는다
5432	PostgreSQL	❌ 차단	컨테이너 간 통신 전용
8080	Spring Boot	❌ 차단	Nginx 프록시 경유만

> **OPC UA·MQTT·DB 포트를 인터넷에 여는 것은 그 자체가 보안 결함이다.** 외부 진입점을 443 하나로 좁히고, 나머지는 Docker 내부 네트워크에 가둔다. 실선 운영 시에는 `edge-gw` ↔ `broker` 구간만 8883(MQTT over TLS)으로 공개한다.

4.2 실선 적용 시 배치 변경

본 프로젝트는 시연을 위해 전 구성요소를 한 VM에 둔다. 실선 적용 시 `sim-fleet` · `edge-gw` 두 컨테이너를 본선으로 이동하고, 시뮬레이터 자리에 실제 AMS 를 연결한다. **나머지 구성과 코드는 변경되지 않는다.**



5. 데이터 흐름

5.1 정상 수집

```
1. sim-fleet    1초 주기로 물리모델 계산 → OPC UA 변수 갱신
2. edge-gw      OPC UA 구독 콜백으로 변경값 수신
3. edge-gw      Deadband 판정 — 변화량이 임계 미만이면 폐기 (전송 생략)
4. edge-gw      전송 대상만 배치로 묶어 MQTT 발행 (QoS 1)
5. broker       토픽 중계
6. ingest       페이로드 스키마 검증 → 불량은 telemetry_rejected 적재
7. ingest       200건 또는 1초 경과 시 telemetry 배치 INSERT
8. ingest       동시에 WebSocket 으로 접속 중인 브라우저에 푸시
9. api          주기 실행되는 알람 룰 엔진이 최신값 평가 → alarm_event 기록
```

5.2 통신 단절 및 복구

```
[단절]
1. edge-gw 가 MQTT 발행 실패 감지
2. 전송 대상을 로컬 큐에 적재 (FIFO, 용량 상한 도달 시 오래된 것부터 폐기)
   ※ 현재 버전은 메모리 큐다. 디스크 저장은 차기 과제 (06-function-spec.md 9장)
3. OPC UA 수집은 계속 진행 — 본선 내부 통신은 영향 없음
4. 대시보드는 해당 선박의 최종 수신시각이 경과함을 표시

[복구]
5. MQTT 재연결 성공
6. 큐를 시간 순서대로 재전송 (실시간 데이터보다 우선)
7. ingest 가 적재 시도 → (tag_id, ts) 유니크 제약으로 중복분 자동 무시
8. 차트의 빈 구간이 소급 채워짐
```

> 중복 제거를 애플리케이션이 아니라 **DB 제약으로 처리**한 이유: 재전송 경로가 여러 개(Edge 큐, MQTT QoS 1 재전송, 수동 재적재)여서 애플리케이션에서 모든 경로를 막으려면 누락이 생긴다. 최종 방어선을 DB 에 둔다.

5.3 적응형 전송 — 모드 전환과 소급 확보

```
[정상 모드]
1. edge-gw  OPC UA 로 1초 원본 수집
2. edge-gw  1초 원본을 Pre-trigger 링버퍼에 적재 (최근 10분 유지, 전송 안 함)
3. edge-gw  1분 경과 시 요약(avg/min/max/sd/n) 산출 → P2 큐로 발행
            └ 평균 약 4.6 kbps

[트리거 — 이상 징후 감지]
4. edge-gw  수집값이 전환 조건 충족 (경보값 80% 근접 / 편차 60% / 변화율 초과 / 품질 저하)
5. edge-gw  고해상도 모드로 전환 (판정 5초 이내, NFR-11)
6. edge-gw  ★ 링버퍼의 직전 10분 원본을 P1 큐로 소급 발행 (buffered=true, mode=raw)
7. edge-gw  이후 1초 원본을 실시간 발행
8. ingest   소급분과 실시간분을 동일 경로로 적재. (tag_id, ts) 중복은 자동 무시
            └ 트리거 이전 구간까지 1초 해상도 확보 (NFR-09)

[복귀]
9. edge-gw  모든 조건 해소 후 정상 10분 지속 시 요약 모드로 복귀

[알람 — 모드와 무관]
*  알람 이벤트는 항상 P0 큐로 즉시 단독 발행 (NFR-10, 3초 이내)
```

이 설계의 요점은 "빨리 전환하는 것"이 아니라 "이미 찍어둔 것을 꺼내는 것"이다. 고장 진단에서 가장 중요한 구간은 이상이 확정되기 **이전**인데, 전환 속도에만 의존하면 그 구간을 영구히 잃는다. 링버퍼를 두면 전환이 다소 늦어도 진단에 필요한 데이터가 보존된다. 오실로스코프의 pre-trigger, 차량 블랙박스의 사고 전 영상과 같은 원리다.

---

6. 기술 선정 근거

6.1 대안 비교

영역	선택	검토한 대안	선택 이유
시계열 저장	**PostgreSQL + TimescaleDB**	InfluxDB, TimescaleDB 전용, Elasticsearch	마스터와 시계열을 **일반 SQL 조인**으로 다룰 수 있음. 운영할 DB 가 하나
선박-육상 전송	**MQTT**	HTTP 폴링, AMQP, WebSocket	저대역·간헐 단절 환경 표준. QoS 1·세션 보존이 규격에 포함
본선 수집	**OPC UA (node-opcua)**	Modbus TCP, 제조사 전용 API	태그 브라우징·인증·읽기전용 권한이 규격에 있어 선급 승인에 유리
OPC UA 서버	**node-opcua 직접 구현**	KEPServerEX, Matrikon, open62541	변환할 이기종 PLC 가 없음. KEPServer 는 유료
업무 API	**Spring Boot 3.5 / Java 21**	Node.js 단일 스택, .NET	트랜잭션·권한·도메인 모델 표현. Virtual Thread 로 I/O 동시성 확보
배포	**Docker Compose / 단일 VM**	Kubernetes, 서버리스	무료 자원 1대. K8s 는 이 규모에서 관리비용이 이득을 초과

6.2 라이선스 검토

구성요소	라이선스	상업적 사용
node-opcua 2.186	**MIT**	가능
Eclipse Mosquitto	EPL 2.0 / EDL 1.0	가능
PostgreSQL 16	PostgreSQL License	가능
TimescaleDB (커뮤니티)	Timescale License	가능 (본 프로젝트 사용 기능 전부 무료 범위)
Spring Boot 3.5	Apache 2.0	가능
Docker Engine / Nginx	Apache 2.0 / BSD-2	가능
AWS EC2 (무료 플랜)	가입 크레딧 100달러, 6개월	월 약 80달러를 크레딧에서 차감. 크레딧 소진 시 청구 없이 정지
총 라이선스 비용	**0원**	

> **실무에서 OPC UA 비용은 클라이언트가 아니라 서버 측에서 발생한다.** AMS·SCADA 제조사가 OPC UA 서버 기능을 옵션 라이선스로 판매하는 경우가 많아, 프로젝트 착수 전 **대상 설비의 OPC UA 서버 라이선스 활성화 여부 확인**이 선행되어야 한다.

---

7. 보안 설계

영역	조치
외부 노출	443 단일 진입점. 나머지 포트는 Docker 내부망에 격리
전송 구간	선박↔육상 MQTT 구간은 TLS(8883)만 받는다. 평문 리스너는 없다. HTTPS(Let's Encrypt)는 차기
수집 구간 인증	사설 CA 가 발급한 **선박별 클라이언트 인증서**(상호 TLS). 브로커가 인증서의 IMO 번호로 발행 토픽을 제한한다 (ADR-011)
인증	JWT. 만료 120분, 서명키는 환경변수 주입
권한	ADMIN / SHORE / CREW 역할 분리. CREW 는 소속 선박으로 행 수준 제한
설비 보호	**OPC UA 세션을 읽기 전용으로 생성. Write·Method 호출 경로 자체를 두지 않음**
비밀정보	`.env` 는 저장소에서 제외. 운영값은 서버에만 존재
DB 계정	애플리케이션 계정과 관리 계정 분리

> 선박 시스템에서 **감시계와 제어계의 분리**는 기본 원칙이다. IACS UR E26/E27(선박 사이버보안, 2024-07 이후 계약 신조선 적용)을 고려하여 본 시스템은 설계 단계에서 쓰기 경로를 배제했다.

---

8. 확장성·가용성

항목	현재	확장 방향
선박 수	4척 (143 값/초)	`edge-gw` 를 선박당 1개로 늘림. 육상은 동일

항목	현재	확장 방향
수집 처리량	단일 `ingest`	MQTT 공유 구독(Shared Subscription)으로 수평 확장
조회 부하	단일 `api`	무상태(stateless)라 인스턴스 증설 가능
DB	단일 인스턴스	읽기 전용 복제본 분리
장애 복구	컨테이너 `restart: unless-stopped`	헬스체크 기반 자동 재기동
데이터 보호	일 1회 논리 백업	오브젝트 스토리지 전송

9. 주요 설계 결정 기록 (ADR)

ADR-001 — Node.js 와 Java 를 함께 사용한다

- **상황** 단일 언어로 통일하면 관리가 단순하다. 그럼에도 두 스택을 쓴다.
- **결정** 수집 계층은 Node.js, 업무 계층은 Java/Spring Boot 로 한다.
- **근거** 두 계층의 문제 성격이 다르다. 수집은 다수 연결과 처리량이 지배하는 **I/O 문제**이고, 업무는 도메인 복잡도와 트랜잭션 정합성이 지배하는 **모델링 문제**다.
- **근거(수집과 업무를 다른 프로세스로 나누는 이유)**
  - 감시 시스템은 **수집이 멈추면 안 된다.** 업무 API 는 기능 추가로 자주 재배포하는데, 한 프로세스에 두면 배포할 때마다 수집이 끊긴다.
  - 조회 부하나 API 결함으로 업무 계층이 느려지거나 죽어도 수집·적재는 계속된다(장애 격리).
  - 본선 Edge 는 저사양 장비라 Node.js 로 만든다. 육상 수집기도 같은 언어로 두어 메시지 형식·검증 코드를 한 언어로 맞춘다.
  - 수집량이 늘면 수집기만 따로 늘릴 수 있다.
- **버린 대안 1** Node.js 단일 스택 — 트랜잭션·권한 로직의 타입 안정성이 떨어진다.
- **버린 대안 2** Java 단일 스택(Spring Boot 안에서 MQTT 구독·적재·WebSocket 까지 처리) — 동작은 한다. 작은 팀·소규모라면 오히려 운영이 단순해 합리적이다. 다만 위의 배포 시 수집 중단·장애 전파 문제가 생기고, 본선 Edge 와 언어가 갈린다.
- **결과** 두 서비스가 DB 로만 결합되어 독립 배포가 가능해졌다. 대신 빌드 파이프라인이 둘로 늘었고, 언어가 둘이라 운영·학습 부담이 커졌다.

ADR-002 — 시계열 전용 DB 대신 TimescaleDB 를 쓴다

- **상황** 초당 143건의 시계열이 발생한다. InfluxDB 등 전용 TSDB 가 일반적 선택이다.
- **결정** PostgreSQL + TimescaleDB 확장으로 마스터와 시계열을 한 DB 에 둔다.
- **근거** "STELLA 선급의 3번 실린더 배기온도" 같은 질의가 **마스터와 시계열의 조인**이다. DB 를 분리하면 이 조인을 애플리케이션에서 수동 결합해야 한다.
- **버린 대안** InfluxDB — 조인 불가, 운영 대상 DB 2개 증가.
- **결과** 표준 SQL 로 해결된다. 단 TimescaleDB 의 연속집계·압축 동작을 이해해야 한다.

ADR-003 — 시뮬레이터를 OPC UA 서버로 구현한다

- **상황** 실선 데이터를 쓸 수 없다. 시뮬레이터가 MQTT 로 바로 발행하면 구현이 단순하다.
- **결정** 시뮬레이터를 **OPC UA 서버**로 만들고, Edge 수집기가 OPC UA 클라이언트로 읽는다.
- **근거** 실선에서 데이터는 AMS 의 OPC UA 서버에서 나온다. 인터페이스를 실제와 같게 두면 **실선 적용 시 엔드포인트 주소 변경만으로 대체**된다. 시뮬레이터가 MQTT 를 직접 쏘면 Edge 계층의 Deadband·버퍼링 설계를 검증할 대

상 자체가 사라진다.

- **버린 대안** ① 시뮬레이터 → MQTT 직접 발행 ② KEPServerEX 도입(유료, 변환 대상 없음)
- **결과** 컴포넌트가 하나 늘었지만, 위성통신 대응 설계를 실제로 시험할 수 있게 되었다.

ADR-004 — 원격 제어 기능을 만들지 않는다

- **상황** 감시 시스템에 제어를 붙이면 기능적으로 완결돼 보인다.
- **결정** 읽기 전용으로 한정한다. OPC UA 세션 권한 자체를 읽기로 제한한다.
- **근거** 기관 제어는 SOLAS·선급 규정 대상이며, 감시계와 제어계 분리가 선박 시스템의 기본이다. "만들 수 있지만 만들지 않는" 판단이 도메인 이해를 보여준다.
- **결과** 사고 시 본 시스템이 원인이 될 경로가 존재하지 않는다.

ADR-005 — 중복 제거를 DB 제약으로 처리한다

- **상황** 재전송 경로가 Edge 큐·MQTT QoS 1·수동 재적재로 여러 개다.
- **결정** `telemetry(tag_id, ts)` 유니크 인덱스 + `ON CONFLICT DO NOTHING`.
- **근거** 애플리케이션에서 모든 경로를 막으면 경로 추가 시마다 누락 위험이 생긴다.
- **결과** 어떤 경로로 들어와도 중복이 생기지 않는다. 삽입 비용이 소폭 증가한다.

ADR-006 — 쿠버네티스를 쓰지 않는다

- **상황** 컨테이너 7개를 운영한다. K8s 가 표준적 선택으로 보인다.
- **결정** 단일 VM + Docker Compose.
- **근거** 무료 자원 1대에서 운영한다. 이 규모에서 K8s 는 제어 플레인 자원과 운영 복잡도가 얻는 이득을 초과한다. `docker compose up` 한 줄로 전체가 재현되는 것이 평가 측면에서도 유리하다.
- **결과** 무중단 배포·자동 스케일링은 포기했다. 확장 시 전환 경로는 8장에 정리했다.

ADR-007 — 적응형 전송과 Pre-trigger 링버퍼를 도입한다

- **상황** 초기 설계는 Deadband 만으로 대역을 줄이고 1초 해상도를 상시 전송하는 방식이었다. 검산 결과 Deadband 적용 후에도 선박당 **월 약 5GB** 가 발생했다. 해상 통신 요금제가 월 데이터량 기준으로 판매된다는 점에서 실선 적용이 어려운 수준이다.
- **결정** 평상시에는 1분 요약(avg/min/max/sd/n)만 전송하고, Edge 가 이상 징후를 감지하면 1초 원본으로 전환한다. 전환과 무관하게 **1초 원본을 10분 링버퍼에 항상 보관**하고, 트리거 시 직전 구간을 소급 전송한다.
- **근거**
  - 정상 운전 중 1초 해상도는 진단 가치가 낮다. 요약에 min/max 를 포함하면 스파이크도 놓치지 않는다.
  - 단순 저속화만 하면 **이상 발생 직전 구간의 해상도를 잃는다.** 진단에 가장 필요한 구간이다.
  - 링버퍼를 두면 보장이 **전환 지연에 의존하지 않는다.** 전환이 늦어도 데이터는 이미 확보되어 있다.
  - 판정을 육상이 아니라 Edge 에서 수행한다. 육상은 요약만 보므로 조기 감지가 구조적으로 불가능하고, 왕복 지연까지 더해진다.
- **버린 대안**
  - ① 상시 1초 전송 — 대역·비용 초과
  - ② 상시 1분 요약만 — 고장 진단 불가
  - ③ 육상에서 판정 후 고해상도 요청 — 감지 지연 + 직전 구간 소실
- **결과** 월 데이터량이 약 5GB → **1.5GB** 로 감소하면서, 진단 구간의 해상도는 오히려 확보되었다. 대신 Edge 에 판정 로직과 링버퍼(약 5MB 메모리)가 추가되어 구현 복잡도가 증가했다. **통신 단절 중에는 어떤 보장도 성립하지 않으며**, 이를 요구사항에 명시했다(NFR-08·NFR-10 단서).

ADR-008 — 업무 API 의 DB 접근에 JPA 대신 JdbcClient 를 쓴다

- **상황** Spring Boot 에서는 JPA 가 기본 선택이다. 초기 설계도 JPA 를 전제로 했다.
- **결정** Spring 6.1 의 `JdbcClient` 로 SQL 을 직접 쓴다.
- **근거** 이 시스템에서 값어치 있는 질의는 `time_bucket` 집계, 계측점별 최신값을 뽑는 `LATERAL` 조인, 부분 유니크 인덱스를 겨냥한 `ON CONFLICT … WHERE` 다. 전부 PostgreSQL·TimescaleDB 고유 문법이고, **SQL 자체가 설계 대상**이다. JPA 로 감싸면 네이티브 쿼리가 대부분이 되어 ORM 을 쓰는 이득이 없다. 엔티티 수가 적고 연관 탐색이 얕아 객체 매핑의 이점도 작다.
- **버린 대안** ① JPA + 네이티브 쿼리 — 두 방식이 섞여 일관성이 떨어진다 ② MyBatis — 가능한 선택이나 XML 매퍼를 둘 만큼 질의가 많지 않다.
- **결과** 질의가 코드에 그대로 드러나 실행계획을 검토하기 쉽다. 대신 행 매핑을 직접 쓴다. 마스터 등록·수정(FR-201~204)이 늘어나면 그 영역에 한해 JPA 도입을 다시 검토한다.

ADR-009 — 알람 판정은 육상 API 에 두고, 본선은 전송 모드만 판정한다

- **상황** 이상 판정을 본선 Edge 와 육상 중 어디에 둘지 정해야 한다. 적응형 전송(ADR-007)은 본선 판정이 필수지만, 알람까지 본선에 두면 룰 관리가 선박마다 흩어진다.
- **결정** 판정을 둘로 나눈다. **"더 자세히 볼 것인가"는 본선**이, **"알람인가"는 육상**이 정한다.
- **근거**
  - 알람 룰은 운영 중에 조정된다. 육상 DB 한 곳에 두면 전 선단에 즉시 반영되고 이력이 남는다. 본선에 두면 위성 구간으로 룰을 배포하고 선박별 버전을 관리해야 한다.
  - 본선 기준을 육상 알람보다 낮게 두므로(편차 30℃ 대 50℃), 알람 판정에 필요한 구간은 이미 1초 원본으로 육상에 와 있다. 육상에서 판정해도 해상도를 잃지 않는다.
  - 알람의 확인·해제·중복 방지는 상태와 이력을 다루는 일이라 DB 제약과 함께 두는 편이 안전하다.
- **버린 대안** 본선에서 알람까지 확정해 전용 토픽으로 발행 — 통신 단절 중에도 알람이 확정된다는 장점이 있다. 규격(`03-interface-spec.md` 4.4)은 남겨 두고 차기 과제로 둔다.
- **결과** 통신이 끊긴 동안에는 알람이 발생하지 않고, 복구 후 버퍼가 들어온 뒤에 판정된다. 본선에는 AMS 자체 경보가 있으므로 본선 승무원의 안전에는 영향이 없다. 본 시스템의 알람은 육상 관제용이다.

ADR-010 — CI/CD 는 GitHub Actions 로 하고, 배포는 서버에서 다시 빌드한다

- **상황** 배포가 수동이다. 서버에 접속해 `git pull` 과 `docker compose up` 을 직접 실행한다. 시험을 돌리지 않은 커밋이 그대로 서버에 올라갈 수 있다.
- **결정** GitHub Actions 워크플로 2개를 둔다.
  - **CI** (`ci.yml`) — push·PR 마다 단위 시험, Node.js 문법 검사, **컨테이너 7개 기동 확인**을 돌린다.
  - **CD** (`deploy.yml`) — CI 를 통과한 커밋만 SSH 로 서버에 반영한다. 서버에서 하는 일은 수동 절차와 같다 (`09-deployment.md` 3.4).
- **근거**
  - 저장소가 GitHub 에 있으므로 별도 서버 없이 쓸 수 있다. 공개 저장소는 실행 비용이 없다.
  - 이 시스템의 결함은 프로그램 사이에서 생긴다 (`08-test.md` 1장). 그래서 CI 에서도 단위 시험만 돌리지 않고 전체를 띄워 선박 4척이 online 이 되는지까지 본다.
  - 수동 절차를 그대로 자동화했으므로, 자동 배포가 고장 나도 같은 명령으로 손으로 배포할 수 있다.
- **버린 대안**
  - Jenkins — CI 서버를 한 대 더 운영해야 한다. 무료 자원 1대라는 전제(ADR-006)와 맞지 않는다.
  - 이미지를 레지스트리에 올리고 서버가 받아 쓰는 방식 — 서버가 여러 대일 때 맞는 방식이다. 한 대에서는 레지스트리 계정·인증·태그 관리만 늘어난다.
- **결과** 서버가 빌드 부하를 진다. 배포 중 다시 만들어지는 컨테이너는 잠깐 끊긴다 (무중단 아님, ADR-006). 되돌릴 때는 이전 커밋으로 되돌린 커밋을 push 한다.

ADR-011 — 선박↔육상 MQTT 구간은 상호 TLS 로 인증하고, 인증서의 IMO 번호로 토픽 권한을 건다

- **상황** 실선에서는 본선 장비가 인터넷(위성) 너머에서 브로커에 접속한다. 익명·평문 브로커라면 누구나 가짜 계측값을 넣을 수 있고, 한 선박이 다른 선박 행세를 할 수 있다. 가짜 값은 곧 가짜 알람이다.
- **결정**
  - 브로커의 평문 리스너를 없애고 TLS(8883)만 연다.
  - 사설 CA 를 두고 **선박마다 클라이언트 인증서**를 발급한다. 인증서의 CN 이 IMO 번호다.
  - 브로커는 우리 CA 가 발급한 인증서가 없으면 접속을 받지 않고, CN 을 사용자명으로 삼아 **자기 IMO 번호의 토픽에만 발행**을 허용한다. 선박에는 읽기 권한을 주지 않는다.
- **근거**
  - 접속 주체가 사람이 아니라 장비다. 비밀번호는 설정 파일에 평문으로 남고 선박 사이에 돌려쓰기 쉽다. 인증서는 장비마다 다르고 개인키가 장비 밖으로 나가지 않는다.
  - 권한을 인증서의 이름에 걸었으므로 브로커에 계정 DB 를 따로 두지 않는다. 선박을 추가할 때 접근 제어 설정을 고칠 필요가 없다.
  - 접속할 수 있는 상대를 우리가 정하므로 공인 CA 가 아니라 사설 CA 가 맞다. 신뢰의 근원이 우리 쪽에 있다.
- **버린 대안**
  - 서버 TLS + 사용자명·비밀번호 — 위의 비밀번호 문제.
  - VPN — 선박마다 터널을 운영해야 하고, 터널 안에서는 선박별 토픽 권한을 나눌 수 없다.
  - 클라우드 IoT 서비스(AWS IoT Core 등) — 선박 수가 늘면 맞는 선택이다. 무료 자원 1대 전제(ADR-006)에서 제외했다. 인증서와 토픽 권한 구조는 그대로 옮겨 갈 수 있다.
- **결과** 인증서 수명 관리가 운영 업무로 생긴다 (`09-deployment.md` 5.6). 폐기 목록(CRL)은 아직 적용하지 않았다. 시연 환경에서는 CA 개인키가 같은 서버의 볼륨에 있다. 운영에서는 CA 를 서버 밖에 둔다.

---

10. 제약 및 가정

구분	내용
제약	클라우드 자원 1대(ARM 4코어·24GB·200GB), 비용 0원 유지
제약	실선 데이터 및 실제 AMS 접근 불가
제약	개발 기간 12일 (2026-10-07 ~ 10-18)
가정	선박당 태그 수는 40개 이내로 유지된다
가정	선사의 업무용 통신 할당량이 **선박당 월 1.5GB 이상**이다. 실제 계약 조건 확인 후 전송 모드 파라미터를 조정한다
가정	AMS 는 OPC UA 서버를 제공하며 읽기 권한 계정을 발급받을 수 있다

---

11. 변경 이력

버전	일자	내용
v1.0	2026-10-08	최초 작성
v1.5	2026-10-16	ADR-001 에 수집·업무 프로세스 분리 근거와 버린 대안(Java 단일 스택), 대가(운영 부담)를 보강

버전	일자	내용
v1.4	2026-10-12	**ADR-011 MQTT 상호 TLS** 추가. 컴포넌트 통신표·포트 정책·보안 설계를 TLS 8883 기준으로 개정
v1.3	2026-10-12	**ADR-010 CI/CD** 추가 (GitHub Actions, 서버 재빌드 방식)
v1.2	2026-10-10	**구현 결과 반영.** ADR-008(JPA 대신 JdbcClient), ADR-009(알람 판정 위치) 추가. 컴포넌트 간 통신표의 DB 접근 방식 정정 및 시연 제어 경로 추가. 단절 버퍼가 현재 메모리 큐임을 명시
v1.1	2026-10-08	**ADR-007 적응형 전송·Pre-trigger 링버퍼** 추가. 5.3 절(모드 전환·소급 확보 흐름) 신설. `edge-gw` 책임 범위 확대(FR-101~110). 제약·가정의 대역 항목을 월 데이터량 기준으로 변경